

Module 5 Guided Lab Session 3: Operate the Schema

Field	Value
Module	M5: Entity Framework Core 10 and PostgreSQL
Session	3 of 3
Exercises	6 (Migration discipline), 7 (N+1 fix), 8 (Shadow audit + concurrency), 9 (Bulk archive + soft delete)

Prerequisites Check Before You Start

Your TMS solution must contain:

- A working `TmsDbContext` against PostgreSQL with at least one applied migration (`InitialCreate`)
- At least three `IEntityTypeConfiguration` classes `StudentConfiguration`, `CourseConfiguration`, `EnrollmentConfiguration`
- Modelled relationships from Session 2 `Student ↔ Enrollment` and `Course ↔ Enrollment` with explicit `OnDelete` behaviour
- Pagination and `GroupBy` queries that produce SQL `LIMIT/OFFSET` and `GROUP BY` in the console

If any of these are missing, go back to Session 2. The migration, performance, and design work in this session presumes the schema design from Session 2 holds.

What This Session Is Really About

The schema is designed. Today you operate it like a production team.

The four exercises share one theme: **what changes when real users hit your data layer at the same time and the rows pile up?**

1. **Migration discipline** (Exercise 6) every schema change is code, reviewed before it is applied, with an explainable `Down()`
2. **The N+1 trap** (Exercise 7) the most common production performance bug in EF Core, deliberately produced before being fixed
3. **Audit and concurrency** (Exercise 8) a shadow column for `LastUpdated` that does not pollute your DTOs, and a row version that prevents two staff members from silently overwriting each other's edits
4. **Bulk and soft delete** (Exercise 9) `ExecuteUpdateAsync` for set-based archiving and `HasQueryFilter` for soft-delete semantics that work everywhere `IQueryable` does

Exercise 6: Two migrations, inspected

Schema changes are code. You review them like any other pull request.

Tasks

1. Create a first migration if Exercise 1 did not already cover the full graph; otherwise add a **second** migration for a deliberate change (for example add Year or IsArchived on Enrollment, or a RowVersion on Student for Exercise 8).
2. Record in your notes: what Up() does in plain language.
3. Practice rollback to the previous migration on a **throwaway** database only:

dotnet ef database update PreviousMigrationName --project ... --startup-project ...

Checkpoint: At least two migration files in the project; you can explain Down() for the latest one.

Troubleshooting:

- **Model snapshot conflicts:** One person adds a migration at a time; merge carefully.
- **Production:** Never run unchecked migrations; this lab stays on local or shared dev databases.

Exercise 7: Many round-trips vs shaped query

The N+1 **pattern** means one query loads parents and then **extra queries** load related data per parent. With **lazy-loading proxies** enabled, touching student.Enrollments inside a loop can do that automatically. With **default EF Core**, you still get N+1 if you **write** a query inside the loop so the database still pays the same price.

Part A Intentional N+1 (for learning)

```
var students = await db.Students.AsNoTracking().ToListAsync(cancellationToken);
```

```
foreach (var s in students)
{
    // TODO: Query enrollment count for this student inside the loop (use StudentId).
    // This should produce 1 + N SQL statements. Count them in the log.
    var count = await db.Enrollments
        .AsNoTracking()
        .CountAsync(e => e.StudentId == s.Id, cancellationToken);

    Console.WriteLine($"{s.Name}: {count} enrollments");
}
```

Part B Fix with shaping

Rewrite the query so the database does the counting in a single round trip. Use a **projection** that asks SQL to join students and count their enrollments:

```
// Fix: Single query with projection
var report = await db.Students
    .AsNoTracking()
    .Select(s => new
    {
        s.Name,
        EnrollmentCount = s.Enrollments.Count
    })
    .ToListAsync(cancellationToken);

foreach (var r in report)
    Console.WriteLine($"{r.Name}: {r.EnrollmentCount} enrollments");
```

Why this works: EF translates `s.Enrollments.Count` into a SQL subquery (`SELECT COUNT(*) FROM "Enrollments" WHERE ...`), so the entire result comes back in **one** SQL statement instead of $1 + N$.

Alternative approach using `Include` (loads full enrollment objects heavier but sometimes useful):

```
var students = await db.Students
    .AsNoTracking()
    .Include(s => s.Enrollments)
    .ToListAsync(cancellationToken);

foreach (var s in students)
    Console.WriteLine($"{s.Name}: {s.Enrollments.Count} enrollments");
```

Checkpoint: SQL log shows **one** query (or one query + one subquery), not $1 + N$ separate queries. Write one sentence in your notes explaining what changed.

Troubleshooting:

- **Still seeing $N+1$ queries:** You are still calling `CountAsync` inside the loop. The projection must happen inside `Select` so EF can translate it to SQL.
- **Include does not load enrollments:** The navigation property `Enrollments` must be declared as `public ICollection<Enrollment> Enrollments { get; set; }` on the `Student` entity and configured in Fluent API.
- **“Client evaluation” warning on .Count:** You may have a method call EF cannot translate. Use the property `.Count` on the navigation collection (not `Count()` with a predicate EF cannot handle).

Exercise 8: Shadow property audit stamp and concurrency token

Scholarship adjustments need a **LastUpdated** audit column without cluttering every DTO. Concurrent edits should not silently overwrite each other.

Shadow property (audit)

In StudentConfiguration (or OnModelCreating):

```
builder.Property<DateTime>("LastUpdated");
```

// TODO: Set default SQL or update logic e.g. set shadow property before SaveChanges in your service layer:

```
// db.Entry(student).Property("LastUpdated").CurrentValue = DateTime.UtcNow;
```

Run a migration adding the column:

```
dotnet ef migrations add AddStudentLastUpdated --project path\to\infra-project.csproj --startup-project path\to\api-project.csproj
dotnet ef database update --project path\to\infra-project.csproj --startup-project path\to\api-project.csproj
```

Verify the column exists in pgAdmin 4:

1. Open pgAdmin → expand your server → Databases → TmsDb → Schemas → public → Tables → Students → Columns.
2. You should see a LastUpdated column with type timestamp without time zone.
3. Alternative: run this SQL in pgAdmin's Query Tool:

```
SELECT column_name, data_type
FROM information_schema.columns
WHERE table_name = 'Students' AND column_name = 'LastUpdated';
```

Expected: one row showing LastUpdated | timestamp without time zone.

Concurrency

Add a uint Version on Student and configure it in the Fluent API: `builder.Property(s => s.Version).IsRowVersion();`. Migrate.

To test concurrency:

1. Open two browser tabs (or two terminal windows) pointing at the same student record.
2. In Tab A, load the student. In Tab B, load the same student.
3. In Tab A, update the name and save.
4. In Tab B, update the GPA and save.

Expected: Tab B's save throws `DbUpdateConcurrencyException` because the row version changed when Tab A saved.

Troubleshooting:

- **No `DbUpdateConcurrencyException`:** The Version property is not configured as a concurrency token. Confirm `IsRowVersion()` is in your Fluent API config and that the migration added the column.
- **Column type mismatch:** PostgreSQL uses `xmin` as a system column for row versioning. The Npgsql provider maps `IsRowVersion()` to `xmin` automatically you do not need to create a separate column. Check the [Npgsql concurrency docs](#) if your provider version differs.
- **Shadow property `LastUpdated` not appearing:** Migration was not applied. Run `dotnet ef database update` and check pgAdmin again.

Exercise 9: Bulk archive and soft-delete filter

Old enrollments must be archived in bulk. Soft-deleted students should disappear from normal queries but remain recoverable for admins.

Tasks

1. Add `HasQueryFilter` on Student for `!IsDeleted` (if not already). Show normal query vs `IgnoreQueryFilters()` for an "admin restore" scenario.
2. Use `ExecuteUpdateAsync` to set `IsArchived` = true for enrollments older than a cutoff **without** loading all rows.

// **TODO:** *ExecuteUpdateAsync with SetProperty match your property names (Year, EnrolledAt, etc.)*

```
// await db.Enrollments
// .Where(/* your predicate */)
// .ExecuteUpdateAsync(s => s.SetProperty(e => e.IsArchived, true), cancellationToken);
```

What you should see:

- SQL log shows a **single** `UPDATE "Enrollments" SET "IsArchived" = true WHERE ...` statement, **not** one `UPDATE` per row.
- Verify in pgAdmin: `SELECT COUNT(*) FROM "Enrollments" WHERE "IsArchived" = true;` should match the number of enrollments older than your cutoff.

Checkpoint: SQL log shows a single `UPDATE` (or provider-equivalent), not thousands of individual updates.

Troubleshooting:

- **ExecuteUpdateAsync not found:** This requires EF Core 7+. Confirm your Microsoft.EntityFrameworkCore package version is 10.x (it should be if you followed M4 setup).
- **IsArchived column does not exist:** Add an IsArchived property to your Enrollment entity, run a migration, and apply it before using ExecuteUpdateAsync.
- **Query filter hides archived rows:** If you added HasQueryFilter(e => !e.IsArchived), normal queries will exclude archived enrollments. Use IgnoreQueryFilters() in an admin endpoint to see them.
- **IsDeleted soft-delete filter not working:** Confirm the filter is applied in StudentConfiguration and that you are NOT calling IgnoreQueryFilters() in your normal queries.

Optional extension: Compiled models

Large models pay startup cost building the model. EF Core can precompile:

```
dotnet ef dbcontext optimize --project ... --startup-project ...
```

Then wire UseModel(...) per [EF Core compiled models](#). Treat this as **stretch** reading not required for module completion.

Why this session ends here

The schema you designed in Session 2 now behaves like a production database:

- **Every schema change is reviewable.** Up() and Down() are first-class artefacts. A teammate who joins next month can read your migrations and understand the history of the schema without you in the room.
- **The N+1 trap is something you produce on demand and fix on demand.** You can spot it in code review, in a SQL log, or in a slow-endpoint complaint from the registrar and you know exactly which shaping technique fixes it.
- **Audit and concurrency are policy, not accident.** The shadow LastUpdated column tracks every change without cluttering your domain types. The row version stops two graders from silently overwriting each other's grades.
- **Bulk and soft-delete are set-based.** Archiving 50,000 enrollments is one UPDATE statement, not 50,000. Soft-deleted students disappear from the registrar's normal queries automatically; admins can still restore them with one filter override.

This is the persistence skillset M6 builds RESTful APIs on top of, M7 layers caching and rate limiting onto, and M11 tests end-to-end. Make sure the verification list below is fully green before you leave M5.

Session 3 Checkpoint

Before completing the module, confirm all five:

- ☐ At least **two migration files** exist in the project; you can read each Up() and Down() aloud and explain what it does
- ☐ The N+1 demonstration in Exercise 7 Part A produces **1 + N** SQL statements in the log; the fix in Part B produces **1** statement (or 1 + 1 subquery) both are visible in the same console output
- ☐ The LastUpdated shadow column exists on Students (verified in pgAdmin); the row-version concurrency test produces a DbUpdateConcurrencyException for the second save
- ☐ ExecuteUpdateAsync for bulk archive logs a **single** SQL UPDATE statement; the soft-delete HasQueryFilter hides IsDeleted students from normal queries and IgnoreQueryFilters() brings them back

If any item fails, finish it before moving on.

Lab Verification Full Module

Run your TMS against PostgreSQL with SQL logging on and verify each item:

#	Check	Session
1	dotnet ef database update succeeds; TmsDb exists in PostgreSQL with Students, Courses, Enrollments	1
2	All four registrar queries produce shaped SQL (WHERE, GROUP BY, aggregates) not SELECT * then C# filter	1
3	Pagination produces SQL with LIMIT 20 OFFSET ... for any page number	2
4	Top-5 courses query produces SQL with GROUP BY and ORDER BY ... DESC LIMIT 5	2
5	Three IEntityTypeConfiguration classes exist; OnModelCreating only contains ApplyConfigurationsFromAssembly	2
6	OnDelete behaviour is set explicitly in Fluent API; the Course→Enrollment relationship has the chosen behaviour	2
7	At least two migration files exist; you can read Up() and Down() for the latest one	3
8	Exercise 7 Part A produces 1 + N SQL statements; Part B produces 1 (or 1 + 1 subquery) for the same data	3
9	ExecuteUpdateAsync produces one bulk UPDATE; HasQueryFilter hides soft-deleted rows from normal queries	3

